DEV_IMPL_GUIDE - CUST-INT-01 Customer Interaction Timeline

Use with `DD_CUST-INT-01-Customer_Interaction_Timeline-v1.0.md`.

This guide explains how the customer timeline projection is implemented.

The most important rule: CUST-INT is a read model. It does not replace ILM notes, TCK tickets, SALES activities, NOT logs, WO notes, or billing records.

1. Runtime Components

Component	Responsibility
Customer timeline API	Serves Customer 360 and optional self-care timeline reads.
Timeline ingestion worker	Consumes events from ILM, TCK, SALES, NOT, WO, BIL, future ASR/CVM.
Timeline DB schema	Stores normalized timeline events, links, ingest errors, cursors.
RBAC client	Checks staff/customer access to timeline rows.
ILM client	Validates customer/account references where needed.
Source module clients	Used by Backoffice when opening detail, not usually during ingestion.
Kafka/inbox	Receives source module events.

For a medium deployment, run CUST-INT inside ILM/customer-service or shared business-services. Split later only if event volume or ownership requires it.

2. Before Requests Can Work

Create these tables:

```
customer_timeline_event
customer_timeline_link
customer_timeline_ingest_error
customer_timeline_cursor
inbox_event
```

Recommended indexes:

```
(operator_code, customer_id, event_time desc)
(operator_code, account_id, event_time desc)
(operator_code, subscription_id, event_time desc)
(operator_code, source_module, source_event_id) unique
(source_module, status) on ingest errors
```

Seed allowed categories:

```
NOTE
CALL
TICKET
NOTIFICATION
SALES
VISIT
BILLING
SYSTEM
CVM
```

Seed visibility values:

```
INTERNAL_ONLY
STAFF
CUSTOMER_VISIBLE
```

3. Flow 1 - Source Module Publishes Event

Example: TCK creates a ticket.

TCK commits ticket creation in its own DB and writes outbox event:

```
{
  "eventId": "evt_tck_01J_CREATED",
  "eventType": "TicketCreated",
  "operatorCode": "WIK",
  "aggregateType": "Ticket",
  "aggregateId": "tck_01J2SUPPORT001",
  "occurredAt": "2026-05-29T13:15:00Z",
  "payload": {
    "customerId": "cust_01HX3K9M2P5",
    "accountId": "acct_01HZ_KAREN",
    "subscriptionId": "sub_01HZ_FIBER",
    "ticketNumber": "KE-TCK-2026-000001",
    "typeCode": "TECHNICAL_SUPPORT",
    "categoryCode": "NO_INTERNET",
    "sourceChannel": "CALL_CENTER",
    "summary": "Customer reported no internet.",
    "customerVisible": false,
    "actorUserId": "kc_user_csr_07"
  }
}
```

Requirements for source events:

- include `operatorCode`;
- include `customerId` whenever possible;
- include source record id;
- include safe summary;
- include visibility/customer-visible flag;
- never include raw secrets, password, full ID scan text, or full diagnostic dumps.

4. Flow 2 - Timeline Worker Ingests Event

4.1 Worker Receives Event

Execution:

1. Kafka consumer receives event.
2. Insert into inbox table or process with idempotent offset handling.
3. Determine source module and event type.
4. Normalize event using source mapping.
5. Validate required fields.
6. Insert timeline row and links in one DB transaction.
7. Mark inbox processed and update cursor.

4.2 Normalization Example

Input:

```
{
  "eventType": "TicketCreated",
  "operatorCode": "WIK",
  "aggregateId": "tck_01J2SUPPORT001",
  "occurredAt": "2026-05-29T13:15:00Z",
  "payload": {
    "customerId": "cust_01HX3K9M2P5",
    "accountId": "acct_01HZ_KAREN",
    "subscriptionId": "sub_01HZ_FIBER",
    "ticketNumber": "KE-TCK-2026-000001",
    "summary": "Customer reported no internet.",
    "customerVisible": false,
    "actorUserId": "kc_user_csr_07"
  }
}
```

Normalized row:

```
{
  "operatorCode": "WIK",
```

```
  "customerId": "cust_01HX3K9M2P5",
  "accountId": "acct_01HZ_KAREN",
  "subscriptionId": "sub_01HZ_FIBER",
  "sourceModule": "TCK-01",
  "sourceEventId": "evt_tck_01J_CREATED",
  "sourceRecordType": "TICKET",
  "sourceRecordId": "tck_01J2SUPPORT001",
  "eventTime": "2026-05-29T13:15:00Z",
  "category": "TICKET",
  "eventType": "TicketCreated",
  "direction": "INBOUND",
  "channel": "CALL_CENTER",
  "title": "Technical support ticket opened",
  "summary": "Customer reported no internet.",
  "visibility": "STAFF",
  "customerVisible": false,
  "actorUserId": "kc_user_csr_07"
}
```

Link row:

```
{
  "entityType": "TICKET",
  "entityId": "tck_01J2SUPPORT001",
  "linkRole": "SOURCE_RECORD",
  "displayRef": "KE-TCK-2026-000001"
}
```

4.3 Idempotency

Before insert:

```
SELECT timeline_event_id
FROM customer_timeline_event
WHERE operator_code = :operatorCode
  AND source_module = :sourceModule
  AND source_event_id = :sourceEventId;
```

If found, mark inbox processed and do not insert another row.

5. Flow 3 - Ingestion Error Handling

If event cannot be normalized:

1. Do not drop the event silently.
2. Insert `customer_timeline_ingest_error`.
3. Mark inbox processed only if the error is non-retryable.
4. For retryable errors, keep status `PENDING_RETRY`.
5. Alert if backlog grows.

Example:

```
{
  "sourceModule": "NOT-01",
  "sourceEventId": "evt_not_01J_BAD",
  "errorCode": "CUSTOMER_ID_MISSING",
  "errorMessage": "Notification event did not include customerId.",
  "status": "PENDING_RETRY"
}
```

Retry API:

```
POST /internal/customer-timeline/ingest-errors/ctie_01J_BAD_EVENT/retry
Authorization: Bearer <service-token>
```

Execution:

1. Load error row.
2. Re-run normalization with stored payload.
3. If success, create timeline event and mark error `RESOLVED`.
4. If failure remains, increment attempt count.

6. Flow 4 - Backoffice Loads Customer Timeline

6.1 Backoffice Calls API

```
GET /api/customers/cust_01HX3K9M2P5/timeline?limit=50
Authorization: Bearer <jwt>
```

6.2 API Validates Access

1. Validate JWT.
2. Check permission with EM-CFG-03, for example `customer.read`.
3. Check scope:

- customer/account/franchise scope if available;

- operator scope;

- future data masking policy.

4. Query timeline rows by customer id.
5. Exclude `INTERNAL_ONLY` rows unless user has permission such as `customer.timeline.internal_read`.
6. Return paged results.

Response:

```
{
  "customerId": "cust_01HX3K9M2P5",
  "items": [
    {
      "timelineEventId": "cte_01J_CALL_001",
      "eventTime": "2026-05-29T13:15:00Z",
      "category": "TICKET",
      "eventType": "TicketCreated",
      "title": "Technical support ticket opened",
      "summary": "Customer reported no internet.",
      "visibility": "STAFF",
      "sourceModule": "TCK-01",
      "links": [
        {
          "entityType": "TICKET",
          "entityId": "tck_01J2SUPPORT001",
          "displayRef": "KE-TCK-2026-000001"
        }
      ]
    }
  ],
  "nextCursor": null
}
```

6.3 User Opens Source Detail

If user clicks the ticket link, Backoffice calls TCK:

```
GET /api/tickets/tck_01J2SUPPORT001
Authorization: Bearer <jwt>
```

CUST-INT does not provide source detail beyond the safe summary and links.

7. Flow 5 - Self-Care Loads Customer-Visible Timeline

Only enable if product wants a unified customer-facing history.

```
GET /api/customers/me/timeline?visibility=CUSTOMER_VISIBLE&limit=20
Authorization: Bearer <customer-jwt>
```

Execution:

1. Validate customer JWT and account mapping.
2. Force `visibility = CUSTOMER_VISIBLE` regardless of query params.
3. Filter by authenticated customer/account scope.

4. Return only rows where `customer_visible = true`.

Never expose:

- internal ticket comments;
- staff notes;
- failed internal notifications;
- module diagnostics;
- approval comments;
- fraud/security notes.

8. Flow 6 - Backfill Existing ILM/TCK Data

Use bulk ingestion for migration or after adding CUST-INT to an existing environment.

```
POST /internal/customer-timeline/events/bulk
Authorization: Bearer <service-token>
Content-Type: application/json

{
  "operatorCode": "WIK",
  "sourceModule": "ILM-CFG-01",
  "batchRef": "ilm-notes-backfill-2026-06-01-0001",
  "items": [
    {
      "customerId": "cust_01HX3K9M2P5",
      "sourceEventId": "backfill_note_01HX_JANE_AFTER_INSTALL",
      "sourceRecordType": "CUSTOMER_NOTE",
      "sourceRecordId": "note_01HX_JANE_AFTER_INSTALL",
      "eventTime": "2026-02-18T10:30:00Z",
      "category": "NOTE",
      "eventType": "CustomerNoteAppended",
      "title": "Customer note",
      "summary": "Customer reported service working well.",
      "visibility": "STAFF"
    }
  ]
}
```

Rules:

- Use deterministic `sourceEventId` for backfill.
- Batch size should be limited, for example 1000 rows.
- Backfill must be rerunnable.
- Do not import obsolete/superseded notes unless historical visibility is required.

9. SQL Migration Sketch

Conceptual schema:

```
CREATE TABLE customer_timeline_event (
  timeline_event_id text PRIMARY KEY,
  operator_code text NOT NULL,
  customer_id text NOT NULL,
  account_id text,
  subscription_id text,
  event_time timestamptz NOT NULL,
  ingested_at timestamptz NOT NULL,
  source_module text NOT NULL,
  source_event_id text NOT NULL,
  source_record_type text NOT NULL,
  source_record_id text NOT NULL,
  category text NOT NULL,
  event_type text NOT NULL,
  direction text NOT NULL,
  channel text,
  title text NOT NULL,
  summary text NOT NULL,
  visibility text NOT NULL,
```

```
  actor_user_id text,
  actor_display_name text,
  customer_visible boolean NOT NULL DEFAULT false,
  pii_level text NOT NULL DEFAULT 'LOW',
  metadata_json jsonb NOT NULL DEFAULT '{}'::jsonb,
  UNIQUE (operator_code, source_module, source_event_id)
);

CREATE TABLE customer_timeline_link (
  timeline_link_id text PRIMARY KEY,
  timeline_event_id text NOT NULL REFERENCES customer_timeline_event(timeline_event_id),
  entity_type text NOT NULL,
  entity_id text NOT NULL,
  link_role text NOT NULL,
  display_ref text,
  created_at timestamptz NOT NULL
);

CREATE TABLE customer_timeline_ingest_error (
  ingest_error_id text PRIMARY KEY,
  operator_code text,
  source_module text NOT NULL,
  source_event_id text NOT NULL,
  error_code text NOT NULL,
  error_message text NOT NULL,
  payload_json jsonb NOT NULL,
  status text NOT NULL,
  attempt_count int NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL,
  updated_at timestamptz NOT NULL
);
```

Indexes:

```
CREATE INDEX idx_cte_customer_time
  ON customer_timeline_event(operator_code, customer_id, event_time DESC, timeline_event_id DESC);

CREATE INDEX idx_cte_account_time
  ON customer_timeline_event(operator_code, account_id, event_time DESC, timeline_event_id DESC);

CREATE INDEX idx_cte_subscription_time
  ON customer_timeline_event(operator_code, subscription_id, event_time DESC, timeline_event_id DESC);

CREATE INDEX idx_cte_category
  ON customer_timeline_event(operator_code, category, event_time DESC);
```

If volume grows, partition `customer_timeline_event` monthly by `event_time`.

10. Redaction Rules

Before storing `title` and `summary`:

- remove raw ID document numbers;
- remove passwords, PINs, OTPs;
- remove full card/payment sensitive data;
- remove internal stack traces;
- remove full network diagnostic dumps;
- truncate very long free text;
- keep enough context for a CSR to understand what happened.

Example:

Bad:

```
Customer ID 12345678 says router password is 948201 and card number is ...
```

Good:

```
Customer reported account access issue. Sensitive details redacted.
```

11. Error Handling

Case	Behavior
Duplicate source event	No-op success.
Missing customer id	Create ingest error.
Unknown visibility	Create ingest error.
Unsafe summary detected	Redact if possible; otherwise create ingest error.
Timeline read without permission	`403`.
Self-care asks for internal events	Force customer-visible filter or `403`.
Source module detail unavailable	Timeline still loads; source detail panel shows error.

12. Required Tests

Test	Expected result
Ingest TCK event	Timeline event and link inserted.
Re-ingest same event	No duplicate row.
Ingest ILM note event	`NOTE` category row inserted.
Ingest notification dispatched	`NOTIFICATION` row inserted with proper visibility.
Ingest malformed event	Ingest error row inserted.
Backoffice customer timeline read	Returns ordered page.
Customer-visible self-care read	Returns only customer-visible rows.
Internal-only row	Hidden from self-care and unauthorized staff.
Redaction	Sensitive tokens are not stored in summary.
Pagination	No duplicate/missing rows across pages.
Backfill rerun	Idempotent.

13. Implementation Checklist

1. Add timeline schema migrations.
2. Implement normalization mapping per source module.
3. Implement Kafka/inbox consumer.
4. Implement internal ingestion API for early modules/backfill.
5. Implement read APIs for customer/account/subscription timeline.
6. Implement RBAC checks.
7. Implement self-care customer-visible endpoint only if enabled.
8. Implement ingest error queue and retry/ignore APIs.
9. Add indexes and pagination cursor.
10. Add redaction helper and tests.
11. Patch FE-APP-01 Customer 360 to use CUST-INT for unified history after approval.
12. Patch source modules to publish timeline-safe events where missing.

14. What Not To Build In V1

- Do not build a separate CRM case engine here.
- Do not store full source records.
- Do not replace TCK ticket timeline.
- Do not replace ILM notes/interactions.

- Do not add OpenSearch unless PostgreSQL filters are insufficient.
- Do not expose internal timeline to customer self-care.